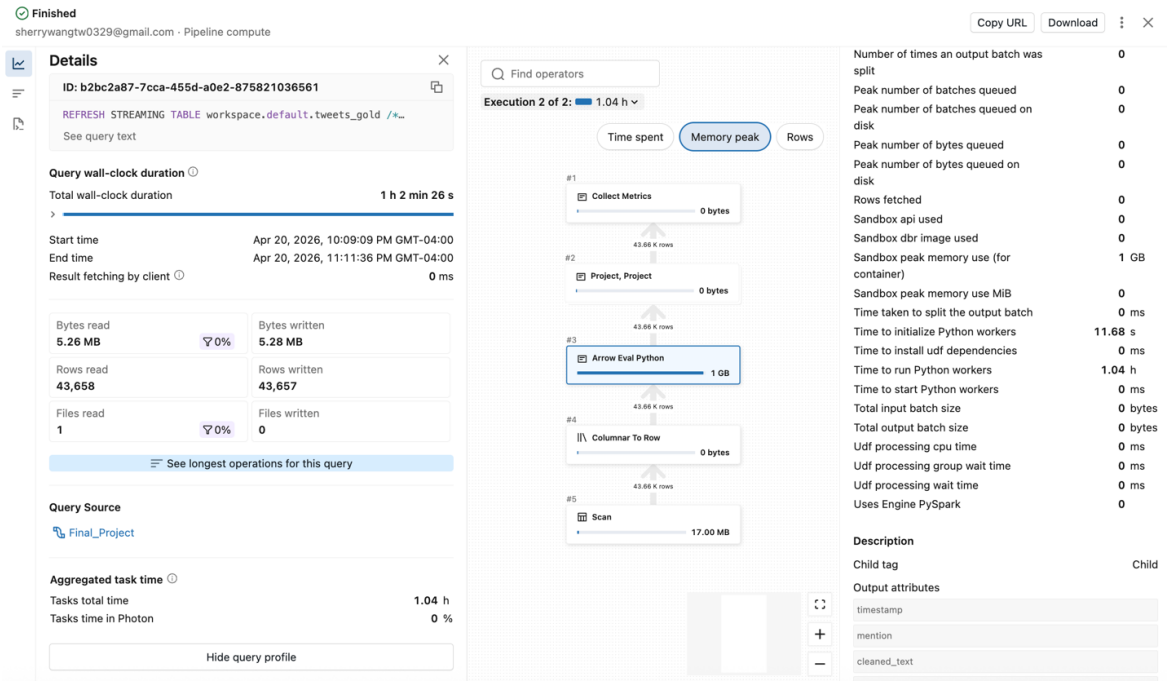
Spark Performance Optimization Analysis

Serialization Analysis



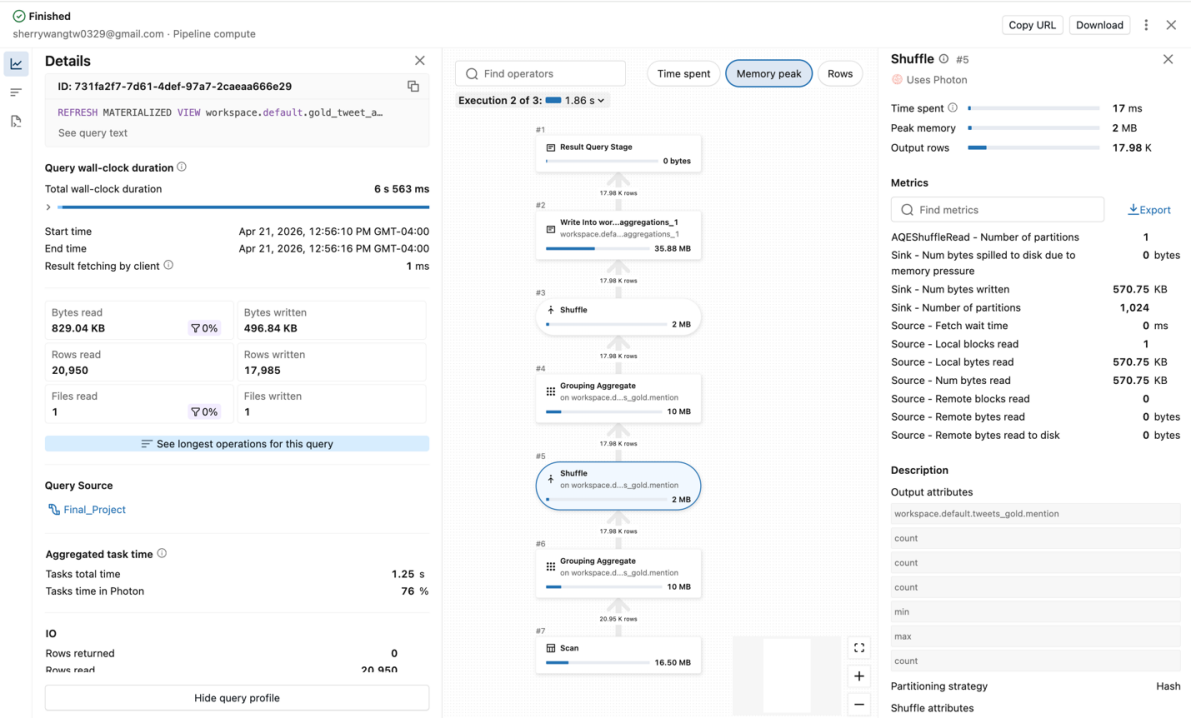
The Spark UI shows that the sentiment prediction stage on the tweets dataset dominates the overall pipeline runtime, with approximately one hour spent in this stage. This stage consumes significant memory (peaking at around 1 GB for 44K rows), primarily due to the use of a Python UDF (Arrow Eval Python), which introduces overhead, as reflected by the ~1.04 hours spent running Python workers. Although this UDF leverages Arrow-based execution, it still requires data to be serialized and transferred between the JVM and the Python runtime, introducing inefficiencies in both memory usage and execution performance. The high memory consumption further suggests substantial cross-language overhead between the JVM and Python processes.

Optimization:

- Replace the current Python UDF with mapInPandas to enable one-time model initialization at the partition level, and use .repartition() to increase parallelism to overcome the compute bottleneck caused by single-file processing.

Shuffle Analysis

The Spark UI shows that aggregation on the mention column introduces shuffle operations. The shuffle data volume is relatively small (571 KB), and no spill to disk is observed, indicating that the current workload is efficiently handled in memory.



However, the original number of shuffle partitions is relatively high (1,024) compared to the small data size (571 KB), which could introduce unnecessary scheduling overhead. In this case, Adaptive Query Execution (AQE) has already optimized the execution by coalescing the partitions down to 1 at runtime, indicating that the dataset is significantly over-partitioned.

Optimization:

- Reduce spark.sql.shuffle.partitions to better match the data size and avoid unnecessary task overhead
- Continue leveraging AQE for dynamic runtime optimization

Spill Analysis

As shown in the same screenshot above, the Spark UI indicates that no spill to disk occurred during the shuffle stage (0 bytes spilled), suggesting that the current workload fits well within available memory. This means that disk I/O overhead caused by memory pressure is not a concern at the current data scale.

Optimization:

- Continuously monitor memory usage as data volume increases, since growing datasets may introduce memory pressure that was not observed in the current run
- Partition tuning may be considered as data scales (e.g., adjusting spark.sql.shuffle.partitions) to balance memory usage and avoid potential spill